

NO REST FOR THE WICKED



JS summarizer

Ricardo G. @ <Academia de Código_>

Table of malcontents

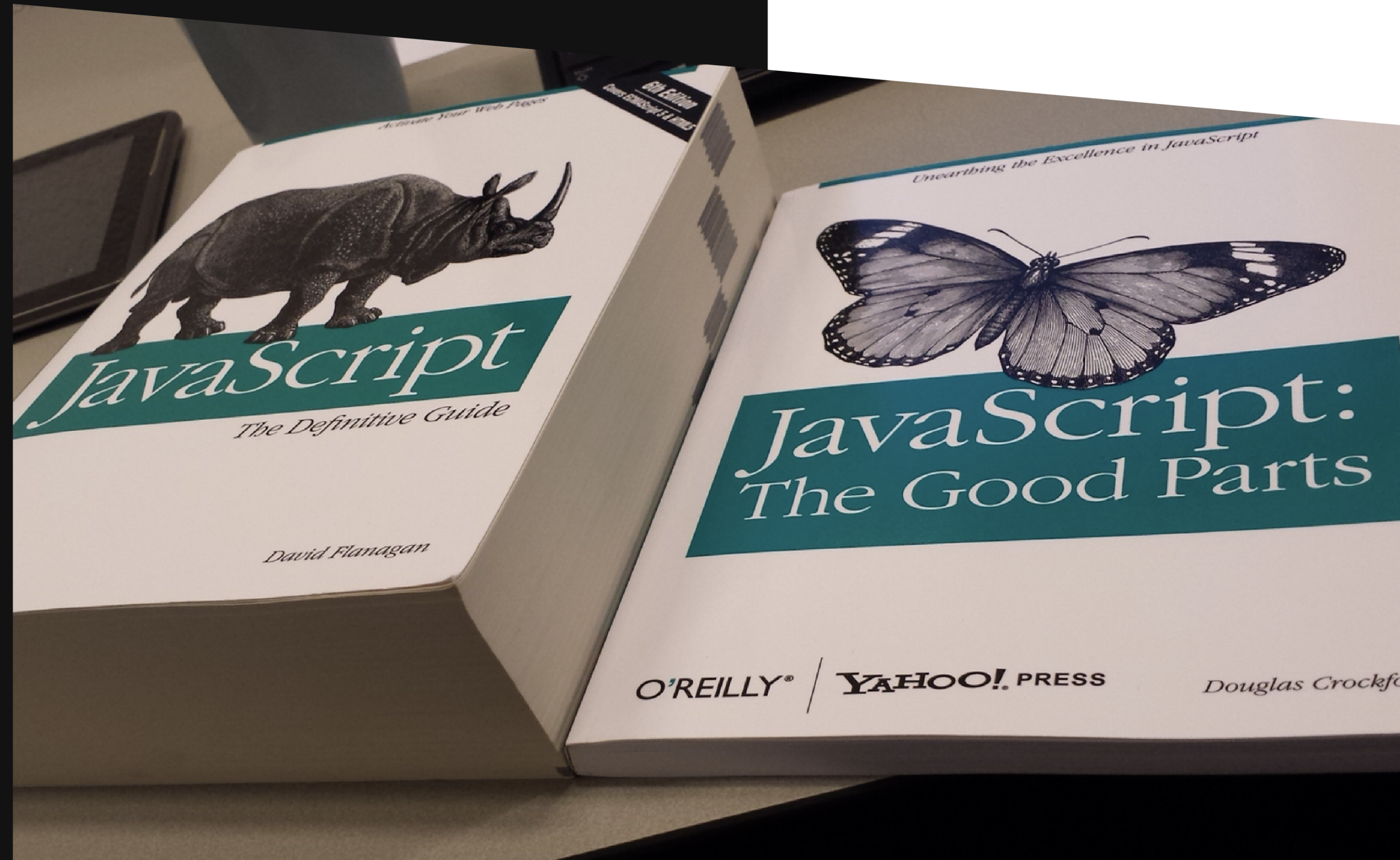
I'm not locked in here with you, you're locked in here with me. ヽ(・`△´)ﾉ”

You're seriously telling me that you can read this tiny text? Good, there's gonna be a looooooottttttttttttt of itttttt hehehehe.

TOPICS:

- Call Stack
- Execution Context
- Variable Enviromnent, Scope and This
- Prototypes
- Closures
- Error Handling

Error handling



It's not about how hard you hit, It's about how hard you can get hit and keep moving forward.

```
try {  
  nonExistentFunction();  
} catch (error) {  
  console.log("batata" +  
error.toString().substring(9));  
}
```

Output to console:

batataError: nonExistentFunction is not defined




```
var data = getData()

try {
  if (!data) {
    throw "Bolo eRRei"
  }

  console.log(data)
  // continue
} catch(error) {
  console.log(error) // "Bolo Rei"
  // Code to redirect to 404 page or
  // any other error page
}
```



AH, THE LEGENDARY BOLO REI REFERENCE...

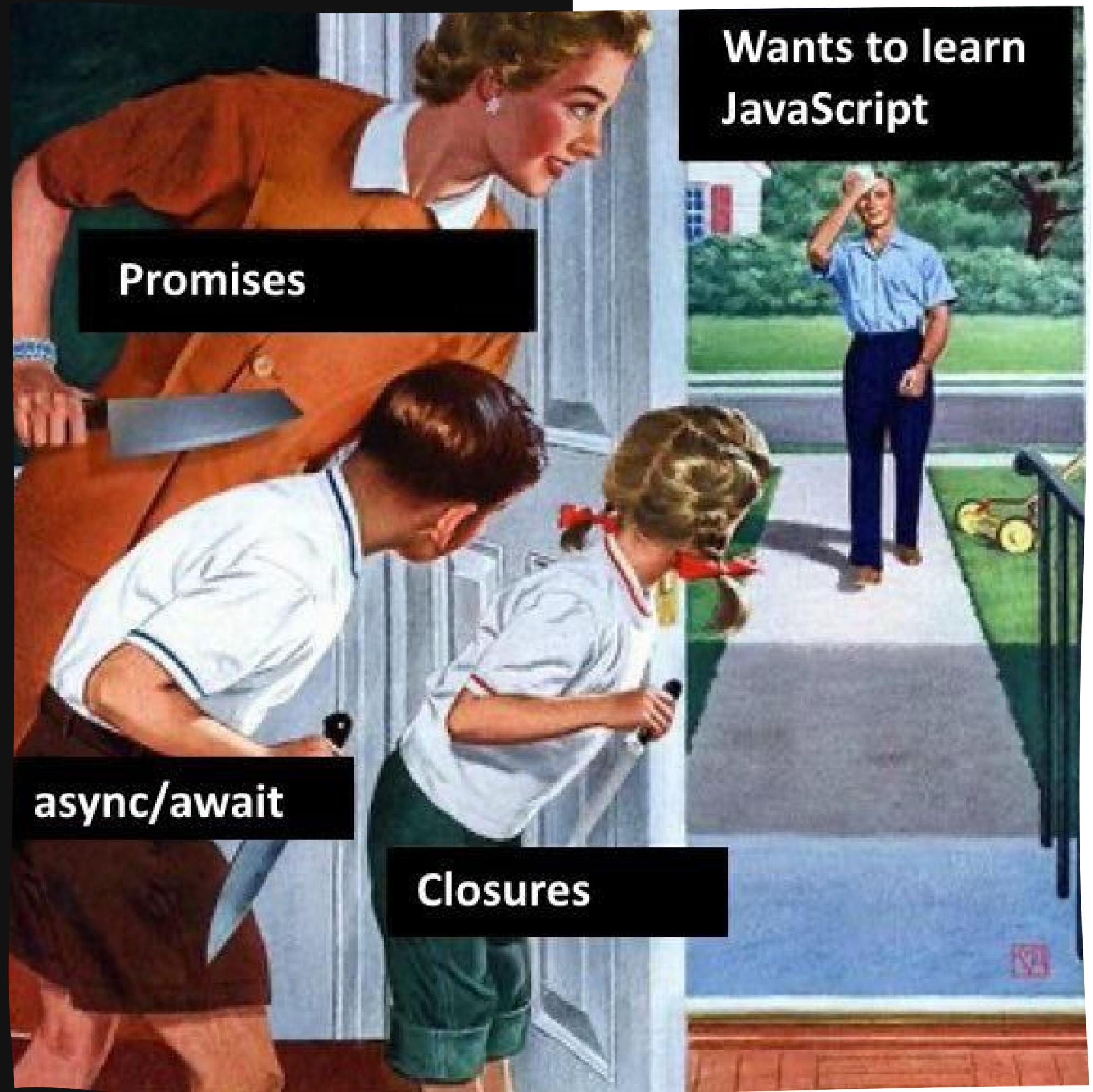
Error Object Properties

- name
- message

Error Name Values

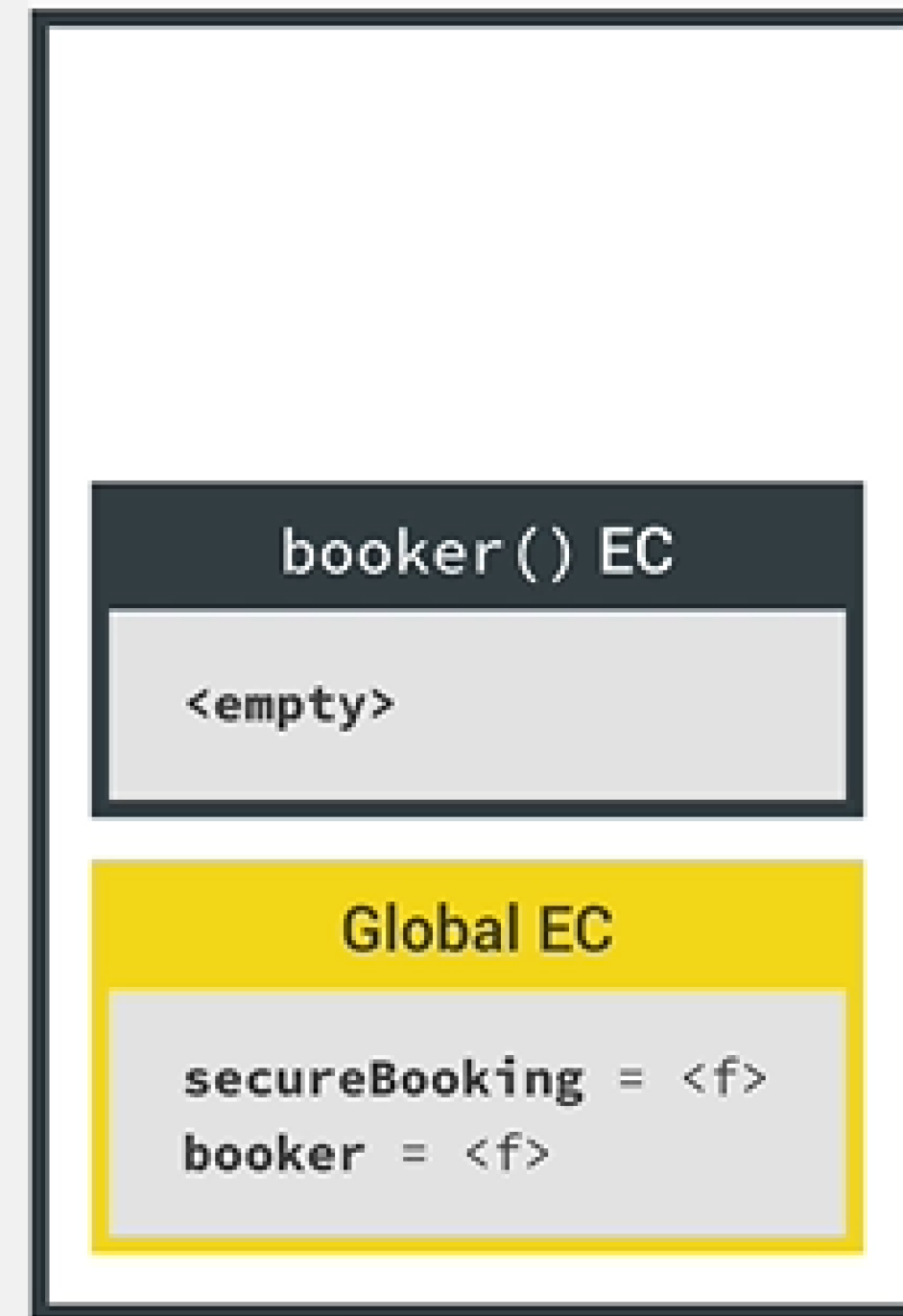
- Error Name
- EvalError
- RangeError
- ReferenceError
- SyntaxError
- TypeError
- URIError

Closures



It keeps coming back

```
var secureBooking = function () {  
  var passengerCount = 0;  
  
  return function () {  
    passengerCount++;  
    console.log(passengerCount);  
  };  
};  
  
var booker = secureBooking();  
booker();  
booker();  
booker();
```



Call Stack

Looking at the clock aren't you?

JUST KEEP THESE 3 MAIN
IDEAS ABOUT CLOSURES
AND YOU'RE GOOD TO
GO... I THINK

1#

A function has access to the EC in
which it was created.

2#

A closure is made when a function
"closes over" its parent scope and
"keeps it for itself".

3#

Closures have priority over the
Scope Chain.

Prototypes

Java Inheritance



JS Pooptrototype



Prototypes and Prototype Chain

01

Every object has a prototype

02

JS looks into the prototype
when it fails to find what its
looking for in the object

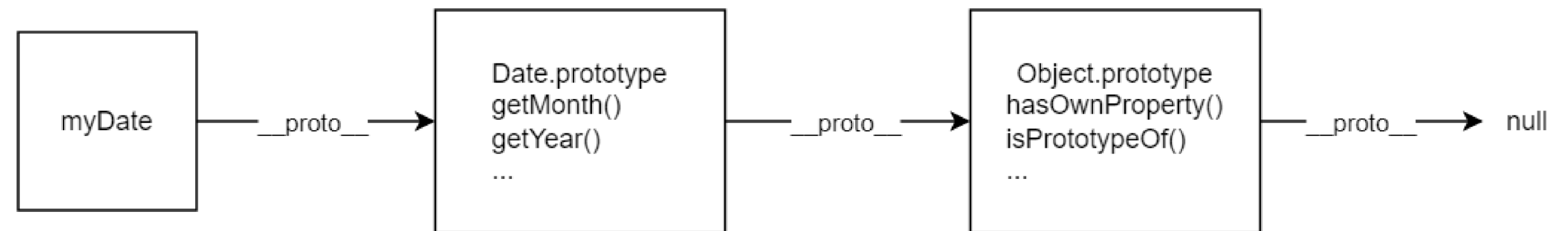
03

All objects ultimately have
`Object.prototype` as the
topmost prototype

Prototypes and Prototype Chain

```
var myDate = new Date();  
var object = myDate;  
  
do {  
  object =  
Object.getPrototypeOf(object);  
  console.log(object);  
} while (object);
```

```
// Date.prototype  
// Object { }  
// null
```



Viewer does not support full SVG 1.1

Variable
Enviromnent

Scope

This





Hint: It's the environment of the variables

WELL, TECHNICALLY

Before execution, code is scanned for variable declarations and, for each variable, a new property is created in the variable environment object.

No Scope Headshot

```
var me = "Ricardo G.";
var occupation = "Trying not to go
over 10-15 mins";
var yearOfBeniginings = 1992;
```

Global Scope

```
function calcAge(birthYear) {
  var present = 2023;
  var age = now - birthYear;
  return age;
}

console.log(now); // ReferenceError
```

Function Scope

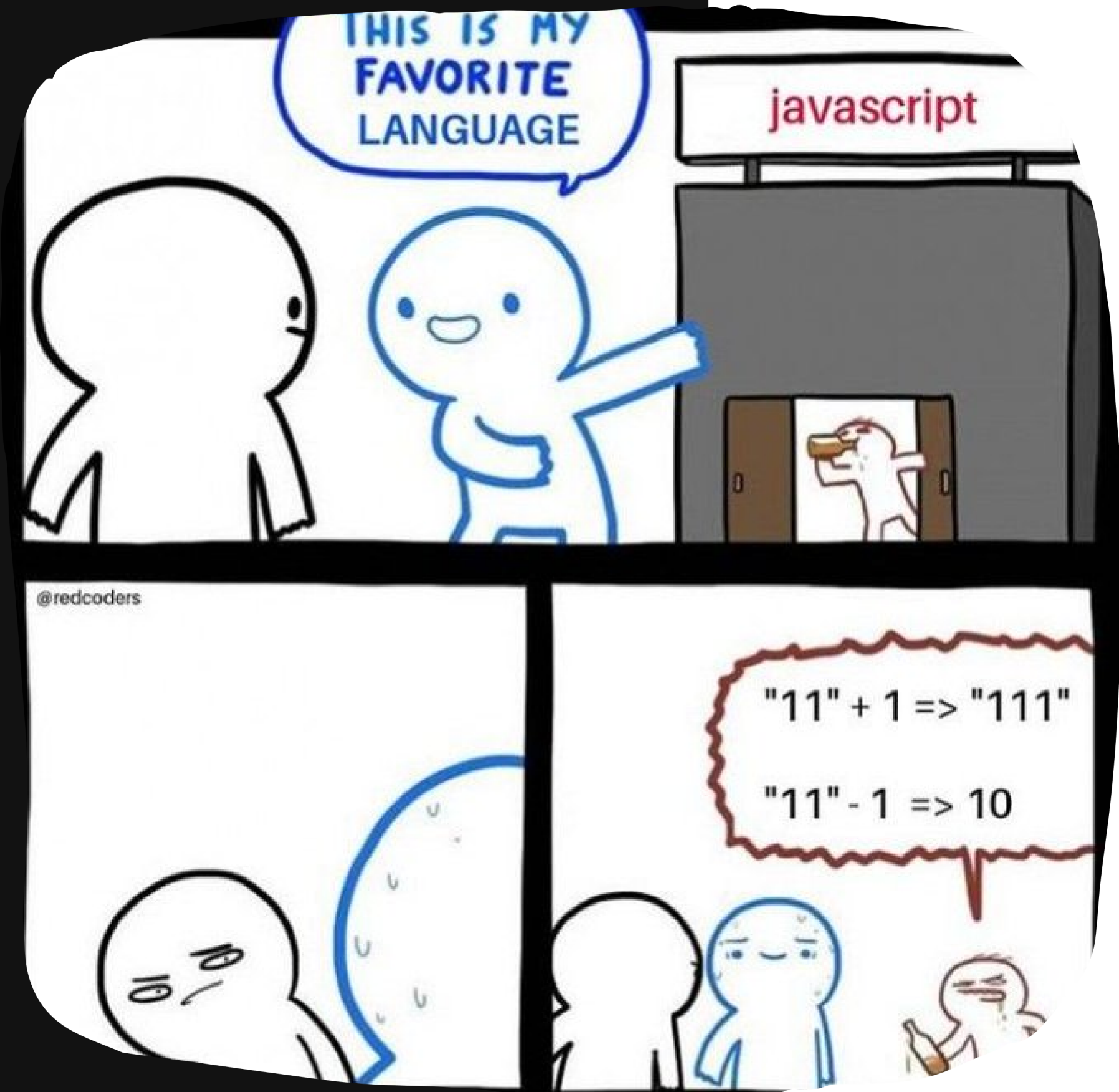
```
if (year >= 1981 && year <= 1996) {
  const millenial = true;
  const food = "Avocado toast";
}

console.log(millenial); //
ReferenceError
```

Block Scope
(ES6, let and const)



this



this

DEFAULT BINDING

```
var exerciciosQueFaltamDeJavaBank = 20;  
  
function batata() {  
  console.log(this.exerciciosQueFaltamDeJavaBank);  
}  
  
batata(); // 20
```



A default binding is made when we do a regular function call

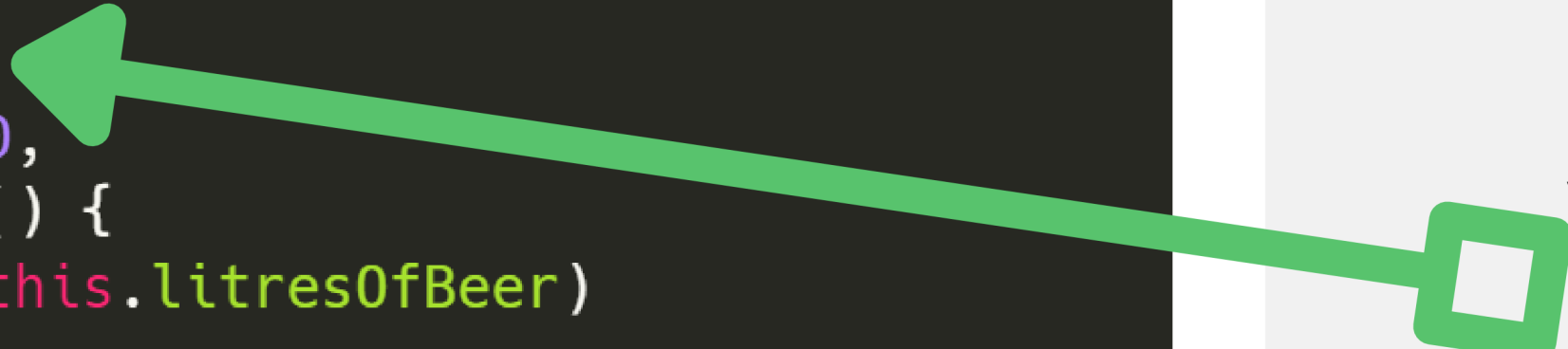
this

IMPLICIT BINDING

```
var litresOfBeer = 20

var codeBreak = {
  litresOfBeer: 50,
  drink: function() {
    console.log(this.litresOfBeer)
  }
}

codeBreak.drink() // 50
```

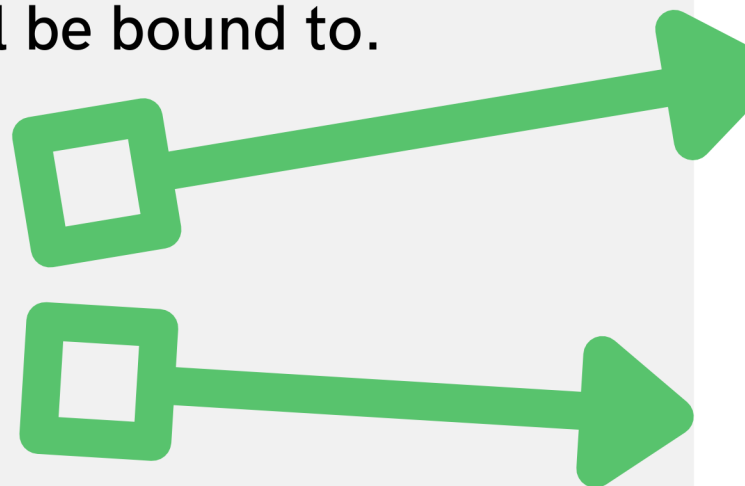


When we invoke a function in the context of an object, this will point to that object.

this

EXPLICIT BINDING

All functions have access to three different methods that allow us to invoke them and explicitly set the object that the this will be bound to.

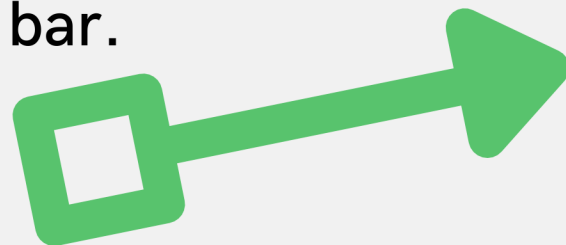


```
var obj = {  
  x: 'Hi there'  
}  
  
function foo(name, age) {  
  console.log(  
    `${this.x}, my name is ${name}, and I'm  
    ${age} years old`  
  )  
}  
  
foo.call(obj, 'Bedro', 31)  
// 'Hi there, my name is Bedro, and I'm 31 years  
old'  
  
var bar = foo.bind(obj, 'Bedro', 31)  
bar() // 'Hi there, my name is Bedro, and I'm 31  
years old'
```

this

NEW BINDING

this will point to the newly created object, which in this case is: bar.



```
function foo(name, age) {  
  this.name = name  
  this.age = age  
}  
  
const bar = new foo('Bedro', 31)  
  
console.log(  
  `My name is ${bar.name}, and I'm ${bar.age} years  
  old`  
)  
  
// My name is Bedro, and I'm 31 years old
```


Execution
Context



Where it all happens

A CONTAINER WITH EVERYTHING NEEDED TO RUN A PORTION OF CODE

One global execution context

Default context for any code not inside a function. (top-level)

One EC per Function

Each function call creates its own Execution Context

All the ECs form the Call Stack

The Call Stack can be seen as the sum of all ECs.

What's inside the ECs?

Variable Environment

- variable declarations
- Functions
- Arguments object

Scope Chain

this Keyword

Execution Context

CREATION PHASE

Global

```
name = 'Ricardo'  
first = <function>  
second = <function>  
x = <unknown>
```

first()

```
a = 1  
b = <unknown>
```

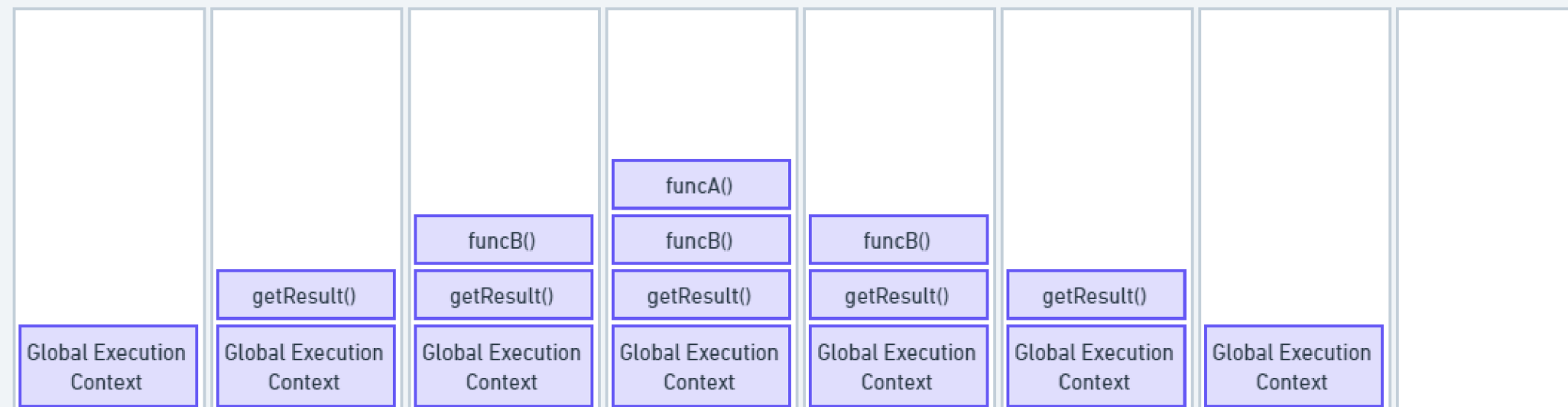
second()

```
c = 2  
arguments = [7, 9]
```

```
var name = "Ricardo";  
  
var first = function() {  
  var a = 1;  
  var b = second(7, 9);  
  a = a + b;  
  return a;  
};  
  
function second(x, y) {  
  var c = 2;  
  return c;  
}  
  
var x = first();
```

EC and Call Stack

EXECUTION PHASE



Call Stack

second()

```
c = 2
arguments = [7, 9]
```

first()

```
a = 1
b = <unknown>
```

Global

```
name = 'Ricardo'
first = <function>
second = <function>
x = <unknown>
```

